TP Synthèse Spring Boot : TalentMatch AI

Table of Contents

1. 1. Vision du Produit . . . 2
   - 1.1. Contexte du TP . . . 2
   - 1.2. Infrastructure Technique . . . 3
2. 2. Le Backlog (User Stories) . . . 3
   - 2.1. Module A : Gestion des Talents et des Opportunités . . . 3
   - 2.2. Module B : Aide à la Décision (IA) . . . 3
   - 2.3. Module C : Sécurité et Fiabilité . . . 3
3. 3. Modèle de Données . . . 4
   - 3.1. 3.1 Entité Candidate . . . 4
   - 3.2. 3.2 Entité JobOffer . . . 4
   - 3.3. 3.3 Entité MatchingResult . . . 5
   - 3.4. 3.4 Note Technique : Persister des List<String> . . . 5
   - 3.5. 3.5 Justification Architecturale . . . 6
4. 4. Spécifications API REST . . . 6
   - 4.1. 4.1 Endpoints Candidates . . . 6
     - 4.1.1. POST /api/candidates . . . 6
     - 4.1.2. GET /api/candidates . . . 7
     - 4.1.3. GET /api/candidates/{id} . . . 7
     - 4.1.4. PUT /api/candidates/{id} . . . 7
     - 4.1.5. DELETE /api/candidates/{id} . . . 7
     - 4.1.6. POST /api/candidates/import . . . 7
   - 4.2. 4.2 Endpoints JobOffers . . . 8
     - 4.2.1. POST /api/job-offers . . . 8
     - 4.2.2. GET /api/job-offers . . . 8
     - 4.2.3. GET /api/job-offers/{id} . . . 9
     - 4.2.4. PUT /api/job-offers/{id} . . . 9
     - 4.2.5. DELETE /api/job-offers/{id} . . . 9
   - 4.3. 4.3 Endpoints Matching . . . 9
     - 4.3.1. POST /api/matching/analyze . . . 9
     - 4.3.2. GET /api/matching/results . . . 10
     - 4.3.3. GET /api/matching/results/{id} . . . 10
     - 4.3.4. GET /api/matching/candidate/{candidateId} . . . 10
     - 4.3.5. GET /api/matching/job/{jobOfferId} . . . 10
   - 4.4. 4.4 Gestion des Erreurs . . . 11

5. 5. Ressources Externes . . . 11
5.1. 5.1 API GitHub . . . 11
5.2. 5.2 Ollama (IA) . . . 12
5.3. 5.3 Kafka . . . 13
6. 6. Contraintes Techniques . . . 14
6.1. 6.1 Architecture . . . 14
6.2. 6.2 Persistance . . . 14
6.3. 6.3 Validation . . . 14
6.4. 6.4 Sécurité . . . 14
6.5. 6.5 Observabilité . . . 14
6.6. 6.6 Tests . . . 14
7. 7. Livrables et Évaluation . . . 15
7.1. 7.1 Ce que Vous Devez Rendre . . . 15
7.2. 7.2 Grille d'Évaluation . . . 15
7.2.1. Palier 1 : Application Fonctionnelle (10/20) . . . 15
7.2.2. Palier 2 : Intégrations Externes (15/20) . . . 16
7.2.3. Palier 3 : Production Ready (20/20) . . . 17
7.3. 7.3 Bonus Appréciés (points supplémentaires) . . . 17
7.4. 7.4 Critères de Qualité Transversaux . . . 18
8. 8. Ressources Utiles . . . 18
8.1. Documentation Officielle . . . 18
8.2. APIs Externes . . . 18
8.3. Outils . . . 19
9. Conclusion . . . 19

1. 1. Vision du Produit

TalentMatch AI est une plateforme conçue pour faciliter la rencontre entre les talents techniques et les besoins des entreprises. Elle permet d'automatiser la création de profils à partir de données réelles (GitHub) et de fournir une analyse intelligente de l'adéquation entre un candidat et une offre d'emploi.

1.1. Contexte du TP

- **Format** : Groupes de 2 à 3 étudiants
- **Durée** : 8h en présentiel (2 séances de 4h) + travail personnel
- **Objectif** : Mettre en pratique l'ensemble des concepts vus en cours
- **Livrable** : Application Spring Boot complète et fonctionnelle

1.2. Infrastructure Technique

Base de données : * H2 en mémoire (pas de PostgreSQL pour simplifier) * Configuration automatique via Spring Boot * Console H2 accessible pour debug

Services externes : * Kafka et Ollama seront fournis et déjà démarrés * Vous n'avez pas à installer ou configurer ces services * Les URLs de connexion vous seront communiquées * Concentrez-vous sur l'intégration applicative, pas sur l'infrastructure

2. 2. Le Backlog (User Stories)

2.1. Module A : Gestion des Talents et des Opportunités

US #1 – Référentiel des Candidats

En tant que recruteur, je veux pouvoir enregistrer et mettre à jour les informations des candidats (compétences, expériences) afin de maintenir une base de talents à jour.

US #2 – Gestion des Offres

En tant que responsable RH, je veux publier des fiches de postes détaillées (missions, technos requises) pour que le système puisse identifier les profils correspondants.

US #3 – Sourcing Automatisé (GitHub)

En tant que candidat, je veux pouvoir pré-remplir mon profil à partir de mon pseudonyme GitHub afin d'éviter la saisie manuelle et de garantir que mes compétences techniques sont basées sur mes projets réels.

2.2. Module B : Aide à la Décision (IA)

US #4 – Analyse d'Adéquation (Matching)

En tant que recruteur, je veux que le système analyse le profil d'un candidat face à une offre précise pour obtenir une synthèse expliquant pourquoi il est (ou n'est pas) le bon choix.

US #5 – Fluidité du Système (Asynchronisme)

En tant qu'utilisateur, je veux pouvoir soumettre une demande d'analyse et continuer à utiliser l'application sans attendre que l'IA ait fini de "réfléchir", car l'analyse peut prendre du temps.

2.3. Module C : Sécurité et Fiabilité

US #6 – Protection des Données

En tant qu'administrateur, je veux que l'accès aux fonctionnalités sensibles (import de profils, analyses IA) soit réservé aux utilisateurs autorisés afin de protéger les ressources du système.

US #7 – Qualité des Données

En tant qu'utilisateur, je veux être alerté immédiatement si les informations saisies sont incomplètes ou invalides afin de garantir l'intégrité de la base de données.

US #8 – Surveillance de la Plateforme

En tant qu'équipe technique, je veux pouvoir vérifier à tout moment si les services externes (IA, bus de données) sont opérationnels pour intervenir rapidement en cas de panne.

3. 3. Modèle de Données

3.1. 3.1 Entité Candidate

Champ	Type	Contraintes
id	UUID	Clé primaire, auto-généré
firstName	String	Obligatoire
lastName	String	Obligatoire
email	String	Obligatoire, unique, format email valide
githubUsernam e	String	Optionnel, unique si renseigné
skills	List<String>	Obligatoire, au moins 1 compétence
yearsOfExperie nce	Integer	Obligatoire, >= 0
bio	Text	Optionnel
createdAt	Timestamp	Auto-généré à la création

3.2. 3.2 Entité JobOffer

Champ	Type	Contraintes
id	UUID	Clé primaire, auto-généré
title	String	Obligatoire
company	String	Obligatoire
requiredSkills	List<String>	Obligatoire, au moins 1 compétence
description	Text	Obligatoire
location	String	Obligatoire
salaryRange	String	Optionnel (ex: "45k-55k")
postedAt	Timestamp	Auto-généré à la création

3.3. 3.3 Entité MatchingResult

Champ	Type	Contraintes
id	UUID	Clé primaire, auto-généré
candidateId	UUID	Obligatoire, référence vers Candidate
jobOfferId	UUID	Obligatoire, référence vers JobOffer
score	Integer	Entre 0 et 100
analysis	Text	Analyse textuelle de l'IA
status	Enum	PENDING, PROCESSING, COMPLETED, FAILED
requestedAt	Timestamp	Auto-généré à la création
completedAt	Timestamp	Renseigné quand status = COMPLETED ou FAILED
errorMessage	Text	Optionnel, renseigné si status = FAILED

3.4. 3.4 Note Technique : Persister des List<String>

Les champs `skills` (Candidate) et `requiredSkills` (JobOffer) sont des listes de chaînes de caractères. JPA ne peut pas mapper directement une `List<String>` sans annotation spécifique.

Solution : @ElementCollection

Cette annotation indique à JPA de créer une table secondaire pour stocker la collection :

```
@ElementCollection
@CollectionTable(name = "candidate_skills", joinColumns = @JoinColumn(name =
"candidate_id"))
@Column(name = "skill")
private List<String> skills;
```

Ce que JPA génère automatiquement :

- Table principale : `candidates` (avec id, firstName, lastName, etc.)
- Table secondaire : `candidate_skills` (avec candidate_id, skill)
- Relation : clé étrangère de `candidate_skills.candidate_id` vers `candidates.id`

Idem pour JobOffer :

```
@ElementCollection
@CollectionTable(name = "job_required_skills", joinColumns = @JoinColumn(name =
"job_offer_id"))
@Column(name = "skill")
private List<String> requiredSkills;
```

Validation de la collection :

Pour garantir qu'au moins une compétence est présente, utilisez `@NotEmpty` sur le champ :

```
@NotEmpty(message = "Au moins une compétence est requise")
@ElementCollection
private List<String> skills;
```

Fonctionne avec H2 : Pas de configuration supplémentaire nécessaire. JPA génère automatiquement les tables.

3.5. 3.5 Justification Architecturale

Pourquoi MatchingResult stocke des IDs au lieu de relations JPA ?

Le matching est géré par une entité dédiée qui stocke les IDs du candidat et de l'offre (pas de `@ManyToOne` ou `@OneToMany`). Cette approche présente plusieurs avantages :

- **Découplage métier** : Les entités Candidate et JobOffer restent indépendantes
- **Historisation naturelle** : Chaque analyse est un événement daté et tracé
- **Évite les pièges JPA** : Pas de LazyInitializationException, pas de stratégies de fetch complexes
- **Performance** : Pas de jointures inutiles lors des requêtes simples
- **Scalabilité** : Permet facilement de déléguer le matching à un service externe ou microservice

Votre responsabilité : Valider l'existence des entités (Candidate et JobOffer) au niveau du service métier avant de créer un MatchingResult.

4. 4. Spécifications API REST

4.1. 4.1 Endpoints Candidates

4.1.1. POST /api/candidates

Créer un nouveau candidat.

Requête :

```
{
  "firstName": "Alice",
  "lastName": "Dupont",
  "email": "alice.dupont@example.com",
  "githubUsername": "alice-dev",
  "skills": ["Java", "Spring Boot", "PostgreSQL"],
  "yearsOfExperience": 3,
  "bio": "Développeuse backend passionnée par les architectures microservices"
```

```
}
```

Réponse : 201 Created

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "firstName": "Alice",
  "lastName": "Dupont",
  "email": "alice.dupont@example.com",
  "githubUsername": "alice-dev",
  "skills": ["Java", "Spring Boot", "PostgreSQL"],
  "yearsOfExperience": 3,
  "bio": "Développeuse backend passionnée...",
  "createdAt": "2024-03-15T10:30:00"
}
```

4.1.2. GET /api/candidates

Lister tous les candidats.

Réponse : 200 OK (tableau de candidats)

4.1.3. GET /api/candidates/{id}

Obtenir les détails d'un candidat.

Réponse : 200 OK (un candidat)

Réponse : 404 Not Found si candidat inexistant

4.1.4. PUT /api/candidates/{id}

Modifier un candidat existant.

Requête : Mêmes champs que POST

Réponse : 200 OK (candidat mis à jour)

4.1.5. DELETE /api/candidates/{id}

Supprimer un candidat.

Réponse : 204 No Content

4.1.6. POST /api/candidates/import

Importer un profil depuis GitHub.

Requête :

```
{
  "githubUsername": "alice-dev",
  "email": "alice.dupont@example.com",
  "firstName": "Alice",
  "lastName": "Dupont"
}
```

Comportement attendu :

1. Appeler l'API GitHub pour récupérer les repositories de l'utilisateur
2. Extraire les langages de programmation utilisés
3. Agréger les compétences (ex: top 5 langages les plus présents)
4. Calculer une approximation de l'expérience (date du premier repository)
5. Récupérer la bio du profil GitHub
6. Créer un candidat avec ces informations

Réponse : 201 Created (même format que POST classique)

Réponse : 502 Bad Gateway en cas d'erreur API GitHub

4.2. 4.2 Endpoints JobOffers

4.2.1. POST /api/job-offers

Créer une nouvelle offre.

Requête :

```
{
  "title": "Développeur Backend Java/Spring",
  "company": "TechCorp",
  "requiredSkills": ["Java", "Spring Boot", "PostgreSQL", "Docker"],
  "description": "Nous recherchons un développeur backend expérimenté...",
  "location": "Paris",
  "salaryRange": "45k-55k"
}
```

Réponse : 201 Created

4.2.2. GET /api/job-offers

Lister toutes les offres.

Réponse : 200 OK

4.2.3. GET /api/job-offers/{id}

Obtenir les détails d'une offre.

Réponse : 200 OK

4.2.4. PUT /api/job-offers/{id}

Modifier une offre existante.

Réponse : 200 OK

4.2.5. DELETE /api/job-offers/{id}

Supprimer une offre.

Réponse : 204 No Content

4.3. 4.3 Endpoints Matching

4.3.1. POST /api/matching/analyze

Lancer une analyse asynchrone entre un candidat et une offre.

Requête :

```
{
  "candidateId": "550e8400-e29b-41d4-a716-446655440000",
  "jobOfferId": "660e8400-e29b-41d4-a716-446655440001"
}
```

Comportement attendu :

1. Valider l'existence du candidat et de l'offre
2. Créer un MatchingResult avec status PENDING
3. Publier un message sur Kafka pour traitement asynchrone
4. Retourner immédiatement une réponse au client

Réponse : 202 Accepted

```
{
  "id": "770e8400-e29b-41d4-a716-446655440002",
  "candidateId": "550e8400-e29b-41d4-a716-446655440000",
  "jobOfferId": "660e8400-e29b-41d4-a716-446655440001",
  "status": "PENDING",
  "requestedAt": "2024-03-15T15:00:00",
  "message": "L'analyse a été soumise et sera traitée prochainement"
```

```
}
```

Réponse : 404 Not Found si candidat ou offre inexistant

4.3.2. GET /api/matching/results

Lister l'historique de toutes les analyses.

Réponse : 200 OK (tableau de MatchingResult)

4.3.3. GET /api/matching/results/{id}

Obtenir le détail d'une analyse spécifique.

Réponse : 200 OK (statut COMPLETED)

```
{
  "id": "770e8400-e29b-41d4-a716-446655440002",
  "candidateId": "550e8400-e29b-41d4-a716-446655440000",
  "jobOfferId": "660e8400-e29b-41d4-a716-446655440001",
  "status": "COMPLETED",
  "score": 78,
  "analysis": "Points forts:\n- Maîtrise solide de Java et Spring Boot\n- Expérience
PostgreSQL confirmée\n\nPoints d'amélioration:\n- Docker : compétence à développer
\n\nRecommandation: Candidat prometteur.",
  "requestedAt": "2024-03-15T15:00:00",
  "completedAt": "2024-03-15T15:00:12"
}
```

Réponse : 200 OK (statut PENDING ou PROCESSING)

```
{
  "id": "770e8400-e29b-41d4-a716-446655440002",
  "status": "PROCESSING",
  "requestedAt": "2024-03-15T15:00:00",
  "message": "L'analyse est en cours de traitement"
}
```

4.3.4. GET /api/matching/candidate/{candidateId}

Lister toutes les analyses d'un candidat spécifique.

Réponse : 200 OK

4.3.5. GET /api/matching/job/{jobOfferId}

Lister toutes les analyses pour une offre spécifique.

Réponse : 200 OK

4.4. 4.4 Gestion des Erreurs

Tous les endpoints doivent renvoyer des réponses d'erreur structurées :

```
{
  "code": "VALIDATION_ERROR",
  "message": "Erreur de validation des données",
  "details": {
    "email": "Email invalide",
    "skills": "Au moins une compétence est requise"
  },
  "timestamp": "2024-03-15T10:30:00"
}
```

Codes d'erreur attendus :

- `VALIDATION_ERROR` (400) : Données invalides
- `NOT_FOUND` (404) : Ressource inexistante
- `GITHUB_IMPORT_ERROR` (502) : Erreur API GitHub
- `AI_SERVICE_ERROR` (503) : Service IA indisponible
- `INTERNAL_ERROR` (500) : Erreur interne

5. 5. Ressources Externes

5.1. 5.1 API GitHub

Endpoint : `GET https://api.github.com/users/{username}/repos`

Headers requis :

```
Accept: application/vnd.github.v3+json
User-Agent: TalentMatch-AI
```

Pas d'authentification requise (limite : 60 requêtes/heure)

Réponse : Tableau de repositories avec notamment :

```
[
  {
    "name": "mon-projet",
    "language": "Java",
    "created_at": "2020-05-15T10:30:00Z",
```

```
    ...
  }
]
```

Endpoint profil utilisateur : `GET https://api.github.com/users/{username}`

Réponse : Informations utilisateur incluant la bio

```
{
  "login": "alice-dev",
  "bio": "Développeuse backend...",
  ...
}
```

Ce que vous devez extraire :

- Liste des langages utilisés (agréger pour avoir les top 5)
- Date du premier repository (pour estimer l'expérience)
- Bio de l'utilisateur

5.2. 5.2 Ollama (IA)

Endpoint : `POST http://localhost:11434/api/generate`

Modèle recommandé : `llama3.2:3b` (léger et rapide)

Requête :

```
{
  "model": "llama3.2:3b",
  "prompt": "Votre prompt ici...",
  "stream": false
}
```

Réponse :

```
{
  "response": "Texte généré par l'IA...",
  "done": true
}
```

Ce que vous devez faire :

1. Construire un prompt structuré qui contient :
    - Informations du candidat (compétences, expérience)

    - Informations de l'offre (compétences requises, description)
    - Instructions pour l'IA (analyser l'adéquation, donner un score/100)
2. Parser la réponse pour extraire :
    - Un score entre 0 et 100
    - Une analyse textuelle (points forts, points faibles, recommandation)

Gestion des erreurs :

- Timeout recommandé : 30 secondes
- En cas d'échec, marquer le MatchingResult comme FAILED avec message d'erreur

5.3. 5.3 Kafka

Topic : `matching-requests`

Workflow asynchrone :

1. **Controller** reçoit POST `/api/matching/analyze`
2. **Service** crée un MatchingResult avec status PENDING
3. **Producer** publie un message sur le topic avec les IDs
4. Réponse immédiate 202 Accepted au client
5. **Consumer** récupère le message
6. **Consumer** charge le candidat et l'offre depuis la DB
7. **Consumer** appelle le service IA Ollama
8. **Consumer** met à jour le MatchingResult (COMPLETED ou FAILED)

Format du message Kafka :

```
{
  "matchingId": "770e8400-e29b-41d4-a716-446655440002",
  "candidateId": "550e8400-e29b-41d4-a716-446655440000",
  "jobOfferId": "660e8400-e29b-41d4-a716-446655440001"
}
```

Pourquoi Kafka ?

L'analyse IA peut prendre 5-15 secondes. Kafka permet de :

- Ne pas bloquer le client HTTP
- Gérer la charge (plusieurs analyses en parallèle)
- Garantir la livraison du message (retry automatique)
- Tracer les événements

6. 6. Contraintes Techniques

6.1. 6.1 Architecture

- **Pattern 3-tiers obligatoire** : Controller → Service → Repository
- **Injection de dépendances** : Par constructeur uniquement
- **Pas de logique métier dans les controllers** : Uniquement validation et délégation

6.2. 6.2 Persistance

- **Base de données** : H2 en mémoire (pour simplifier le setup)
- **ORM** : Spring Data JPA
- **Gestion des relations** : Utiliser des IDs dans MatchingResult (pas de @ManyToOne)

6.3. 6.3 Validation

- **Bean Validation** : Toutes les entités doivent être validées
- **Validation métier** : Vérifier l'existence des entités référencées

6.4. 6.4 Sécurité

- **Endpoints protégés** :
  - `POST /api/candidates/import`
  - `POST /api/matching/analyze`
- **Mécanisme** : Basic Authentication
- **Utilisateurs** : Au moins 2 users (admin, recruiter)

6.5. 6.5 Observabilité

- **Spring Actuator** : Endpoints health, info, metrics activés
- **Custom Health Indicators** :
  - Kafka (vérifier la connexion)
  - Ollama (vérifier la disponibilité)
- **Custom Endpoint** : Statistiques des matchings (total, par statut)

6.6. 6.6 Tests

- **Tests unitaires** : Services avec Mockito
- **Tests web** : Controllers avec MockMvc

- **Tests d'intégration** : Au moins 3 scénarios complets avec Testcontainers

7. 7. Livrables et Évaluation

7.1. 7.1 Ce que Vous Devez Rendre

Code Source :

- Projet Maven complet et fonctionnel
- Tous les packages structurés selon l'architecture 3-tiers
- Configuration `application.yml` ou `application.properties`

Tests :

- Minimum 10 tests unitaires (Mockito)
- Minimum 5 tests web (MockMvc)
- Au moins 3 tests d'intégration (Testcontainers)

Documentation :

- README.md avec :
  - Instructions de démarrage (infrastructure + application)
  - Liste des endpoints avec exemples de requêtes curl
  - Exemples de réponses attendues
  - Utilisateurs de test (username/password pour Basic Auth)
  - Choix techniques et difficultés rencontrées

7.2. 7.2 Grille d'Évaluation

7.2.1. Palier 1 : Application Fonctionnelle (10/20)

Critères obligatoires :

- ☐ Architecture 3-tiers respectée (Controller/Service/Repository)
- ☐ CRUD complet pour Candidate (5 endpoints)
- ☐ CRUD complet pour JobOffer (5 endpoints)
- ☐ Entité MatchingResult créée avec gestion des IDs
- ☐ Validation Bean Validation sur toutes les entités
- ☐ H2 configuré et fonctionnel (mode mémoire)
- ☐ Exception handling centralisé avec @RestControllerAdvice
- ☐ Application démarre sans erreur

- ☐ Au moins 5 tests unitaires

Points de vérification :

- Les entités utilisent UUID comme clé primaire
- Pas de relations @ManyToOne entre les entités (IDs uniquement)
- `@ElementCollection` correctement utilisé pour les listes de compétences
- Messages d'erreur structurés (code, message, timestamp)
- H2 Console accessible pour vérifier les données

Livrables Palier 1 :

- Code source complet
- README avec instructions de démarrage
- Configuration H2 dans application.properties

7.2.2. Palier 2 : Intégrations Externes (15/20)

En plus du Palier 1 :

- ☐ Import GitHub fonctionnel (extraction langages, calcul expérience)
- ☐ Service d'analyse IA avec Ollama (prompt structuré, parsing réponse)
- ☐ Endpoints matching créés (5 endpoints)
- ☐ Gestion des statuts PENDING/PROCESSING/COMPLETED/FAILED
- ☐ Validation métier (existence Candidate/JobOffer avant matching)
- ☐ Gestion des erreurs pour appels externes (timeout, retry, codes HTTP appropriés)
- ☐ Tests MockMvc pour tous les controllers (minimum 10 tests)
- ☐ Au moins 15 tests au total (unitaires + web)

Points de vérification :

- RestTemplate configuré avec timeouts pour GitHub et Ollama
- Parsing de la réponse Ollama extrait score et analyse
- Codes HTTP appropriés (502 pour GitHub, 503 pour Ollama)
- Tests unitaires mockent les appels externes

Livrables Palier 2 :

- Exemples curl complets pour tester GitHub import et matching
- Documentation des choix techniques dans README
- Instructions pour démarrer Ollama localement (si nécessaire)

7.2.3. Palier 3 : Production Ready (20/20)

En plus du Palier 2 :

- ☐ Kafka configuré (producer + consumer)
- ☐ Traitement asynchrone des matchings fonctionnel
- ☐ Message publié sur topic `matching-requests`
- ☐ Consumer traite les messages et met à jour MatchingResult
- ☐ Basic Auth implémenté (minimum 2 utilisateurs)
- ☐ Endpoints sensibles protégés (import, analyze)
- ☐ Actuator avec health indicators custom (Kafka, Ollama)
- ☐ Actuator avec endpoint custom (statistiques matchings)
- ☐ Tests d'intégration avec H2 (minimum 3 scénarios)
- ☐ Au moins 20 tests au total

Points de vérification :

- Le workflow asynchrone fonctionne : POST retourne 202, traitement en background
- Health indicators retournent UP/DOWN selon l'état réel des services
- Basic Auth protège uniquement les endpoints sensibles
- Tests d'intégration testent des scénarios complets end-to-end

Livrables Palier 3 :

- Application complète et robuste
- Documentation architecture (schéma workflow, choix techniques, difficultés)
- Tests exhaustifs avec bonne couverture
- Démonstration du monitoring via Actuator

Note importante : Kafka et Ollama doivent être démarrés localement ou via des services fournis par l'enseignant. L'infrastructure n'est pas à votre charge.

7.3. 7.3 Bonus Appréciés (points supplémentaires)

- Pagination et filtres sur les listes
- Logging structuré avec niveaux appropriés (SLF4J)
- Cache en mémoire (Caffeine) pour les profils GitHub
- Métriques Prometheus exposées via Actuator
- Validation custom avec annotations personnalisées
- Script shell pour tester tous les endpoints
- Documentation technique détaillée avec schémas

- Migration facile H2 → PostgreSQL (configuration externalisée)

7.4. 7.4 Critères de Qualité Transversaux

Code :

- Nommage cohérent et explicite (anglais recommandé)
- Méthodes courtes et responsabilités claires
- Pas de code dupliqué
- Injection de dépendances par constructeur uniquement
- Commentaires uniquement où nécessaire (code auto-documenté)

Git :

- Commits atomiques avec messages explicites
- Pas de fichiers sensibles (`.env`, credentials, `application-local.properties`)
- `.gitignore` correctement configuré

Documentation :

- README complet et structuré
- Instructions de setup étape par étape
- Exemples d'utilisation des endpoints
- Explications des choix techniques et compromis

8. 8. Ressources Utiles

8.1. Documentation Officielle

- Spring Boot : https://docs.spring.io/spring-boot/docs/current/reference/html/
- Spring Data JPA : https://docs.spring.io/spring-data/jpa/docs/current/reference/html/
- Spring Kafka : https://docs.spring.io/spring-kafka/docs/current/reference/html/
- Spring Security : https://docs.spring.io/spring-security/reference/
- Spring Actuator : https://docs.spring.io/spring-boot/docs/current/reference/html/actuator.html

8.2. APIs Externes

- GitHub API : https://docs.github.com/en/rest
- Ollama API : https://github.com/ollama/ollama/blob/main/docs/api.md

8.3. Outils

- curl : Outil en ligne de commande pour tester les API REST
- H2 Console : Accessible via /h2-console
- IntelliJ IDEA : https://www.jetbrains.com/idea/

9. Conclusion

Ce TP de synthèse est l'occasion de démontrer votre maîtrise de Spring Boot et de l'écosystème Spring. Vous devez être autonomes dans vos choix d'implémentation tout en respectant les contraintes architecturales imposées.

Prenez le temps de :

- Bien comprendre les user stories et les besoins métier
- Concevoir votre architecture avant de coder
- Tester régulièrement votre application
- Documenter vos choix techniques

L'objectif n'est pas seulement de livrer une application fonctionnelle, mais de démontrer votre compréhension des concepts Spring Boot et votre capacité à les appliquer dans un contexte réaliste.

Bon courage !